

Scraper Platform Upgrade — Developer Implementation Checklist

Audience: Engineering **Type:** Implementation spec + step-by-step checklist **Scope:** A new version of the existing lead-scraping platform that (1) adds Tennessee as a territory, (2) adds a new vendor/distributor scraper, (3) hardens the lumber-exclusion filter, and (4) introduces a proper data-separation and access layer.

This is an **additive** release. Existing Florida behavior must keep working unchanged. Nothing about the current Florida scrapers is being removed or replaced.

0. How to use this document

Each workstream below has a **goal**, a **checklist** ([] items to complete in order), and **done-when** acceptance checks. Recommended build order is $A \rightarrow E \rightarrow B \rightarrow C$, with D applied across B and C. Workstream **E (data separation)** should be designed first or in parallel, because it determines the schema everything else writes into — doing it last means a refactor.

A short glossary is at the end. Terms are defined on first use.

1. Current system (context)

The existing platform:

- Is built on **Apify** (the scraping platform).
- Finds **drywall contractors** and **general contractors (GCs)** matching an Ideal Customer Profile (ICP) using keyword search.
- Pulls from three sources: **Google Business / Google Maps**, the **Better Business Bureau (BBB)**, and the **Florida DBPR** (Florida Department of Business and Professional Regulation — Florida's public contractor-license lookup).
- Runs an optional **enrichment** step to fill missing contact data.
- Is currently limited to **Florida** and specific Florida zip codes.

Typical output per record: business name, contact first/last name, phone, and general business details from the Google Business profile.

2. What we are building (summary)

1. **Territory expansion** — add Tennessee (states + zip codes), built on configurable geography rather than hardcoded locations.
 2. **Vendor scraper** — a new mode that finds drywall-material **distributors** (the sell-to targets), not individual contractors.
 3. **Lumber exclusion** — a robust, multi-layer filter to remove lumber-focused businesses, which the current keyword-only approach fails to exclude.
 4. **Data separation & access layer** — stage-based, tagged, multi-territory/multi-type data storage with a clean read/export layer.
-

Workstream A — Geography & territory foundation

Goal: Replace hardcoded location logic with configurable geography that supports multiple states, a hard territory exclusion, and city prioritization.

- ☐ Add **Tennessee** as a supported state alongside Florida (additive — Florida config untouched).
- ☐ Move location targeting into **editable configuration / reference tables** (state, city, zip, county) rather than hardcoded values.
- ☐ Implement a **region include/exclude list** (county- or region-based) that is read at run time and editable without code changes.
- ☐ Implement the **hard territory exclusion: Memphis and its metro** (Memphis, Bartlett, Germantown, Collierville, and surrounding far-southwest TN). Records in this region must never be scraped or returned. Everything else in Tennessee — including East TN (Knoxville, Chattanooga, Pigeon Forge) — is in-territory.
- ☐ Implement **two independently-configurable search radii**: vendor radius (**default 20 miles**) and contractor radius (**default 50 miles**).
- ☐ Build a **geocoding step** that converts vendor account addresses to coordinates.
- ☐ For the contractor pipeline: given vendor account locations, compute all **zip codes within the contractor radius**, de-duplicate them, then **apply the territory exclusion before scraping** so we never spend runs on excluded zips.

City prioritization (key feature):

- ☐ Define **Tier 1 — named priority cities**: Pigeon Forge, Nashville, Chattanooga, Jackson, Columbia, Knoxville. These are prioritized **regardless of population** — do **not** size-filter them (Pigeon Forge and Columbia are under 50k but are explicitly required).
- ☐ Define **Tier 2 — all other Tennessee cities with population $\geq 50,000$** , using a **precomputed static list** (city + center coordinates + radius). Prefer a static list over a live population API for predictability; refresh it manually when needed.
- ☐ Implement prioritization as **both**: (a) run-order — Tier 1 scraped before Tier 2, and (b) a **tier tag** on every record so output can be sorted/filtered by priority.

Done when: a run can be scoped to TN, automatically skips the Memphis metro, scrapes Tier 1 cities first, and tags every record with city + tier + zip — all driven by editable config.

Workstream B — Contractor scraper (Tennessee)

Goal: Extend the existing contractor scraper to Tennessee without changing its proven search logic.

- ☐ **Do not modify** the existing keyword/ICP search logic. Reuse it as-is.
- ☐ Feed it the **TN in-territory zip set** produced in Workstream A.
- ☐ Add a **Tennessee contractor-license verification source**. Note Tennessee does **not** publish a Florida-style bulk downloadable list. Use, in priority order:
 - ☐ **Municipal open data** where available — e.g. Nashville's downloadable **"Registered Professional Contractors"** dataset (data.nashville.gov). Check Murfreesboro, Franklin, Knoxville, Chattanooga for equivalents.
 - ☐ **TDCI open-records request** (Tennessee Dept. of Commerce & Insurance, Board for Licensing Contractors) for a baseline licensee roster export.
 - ☐ **verify.tn.gov / search.cloud.commerce.tn.gov** as a **verify-a-name** tool — use for enrichment/validation of names already in hand, **not** for discovery (it returns one record per lookup, not a list).
- ☐ Treat license data as **validation/enrichment**, not the primary discovery engine. Discovery still relies on Google Business + BBB + keywords.
- ☐ Apply the **lumber exclusion** (Workstream D).

Plan around two limits (document, don't try to "solve"):

- Tennessee licensing applies above certain project values; small drywall crews (sub-threshold, no website) won't appear in license data from any source.
- License records are keyed to **trade classification codes**, not the word "drywall" — filter by classification, not keyword.

Done when: a TN run returns contractor records scoped to in-territory zips, optionally cross-referenced against available license data, with lumber businesses filtered out.

Workstream C — Vendor scraper (new feature)

Goal: A new scrape mode that finds drywall-material **distributors**, not contractors.

- ☐ Build the vendor scraper as a **separate mode**, not a modification of the contractor scraper.
- ☐ Anchor searches to the **prioritized city list** (Workstream A), using the **20-mile vendor radius** from each city center.
- ☐ Implement a **vendor alias / subsidiary map** so branches are identified and de-duplicated correctly:

Target	Entity	Identification notes
L&W	L&W Supply (a division of ABC Supply Co.)	May appear as "L&W Supply," "ABC Supply Interiors," or "ABC Supply." Roll up to one network.
FBM	Foundation Building Materials	"FBM" or full name.
GMS	Gypsum Management & Supply	Operates under local subsidiary brand names, not "GMS" — e.g. <i>Drywall Supply, Tucker Materials, Gator Gypsum, Rocky Top Materials</i> . Maintain an alias list or GMS will be badly undercounted.
Independents	Regional dealers	e.g. Dealers Supply, Building Products Supply, and others. Capture by drywall/gypsum category , not a fixed name list.

- ☐ Use the provided seed list ([Nashville_Drywall_Distributors.xlsx](#)) as a **validation/seed set** — confirm and enrich those branches, and discover additional ones. It is unverified, so do not treat it as final output.
- ☐ Output fields prioritize **distributor name** and **branch location**, plus phone and notes (mirror the seed sheet structure).
- ☐ Apply the **lumber exclusion** (Workstream D) and the Memphis exclusion.

Done when: a TN run produces a distributor list across the prioritized cities, with multi-name networks (especially GMS subsidiaries) correctly merged, validated against the seed set.

Workstream D — Lumber exclusion (cross-cutting; known failure point)

Goal: Reliably remove lumber-focused businesses from **both** lists. A single "exclude the word lumber" rule has already proven insufficient — do not rely on keyword exclusion alone.

- ☐ Layer 1 — **Category/type exclusion:** drop records whose source category (Google Maps category, BBB category) indicates lumber/sawmill/lumberyard.
- ☐ Layer 2 — **Negative-keyword list:** maintain an **editable** list (lumber, lumberyard, sawmill, etc.) applied to name and description.
- ☐ Layer 3 — **Name-pattern matching:** catch businesses with a clean category but a lumber-signaling name.
- ☐ Apply all layers to **both** the contractor and vendor pipelines.
- ☐ **Flag, don't hard-delete** (see Workstream E): mark excluded records with an [excluded_reason](#) rather than dropping them, so exclusions are auditable and the "wide net" philosophy is preserved.

Done when: known lumber businesses are reliably absent from both deliverable lists, and every exclusion is recorded with a reason.

Workstream E — Data separation & access layer (new)

Goal: Store scraped data so it can be cleanly separated by client, record type, territory, source, and pipeline stage — and accessed/exported without touching raw data. This is what lets the platform scale to more clients and states without becoming a tangle.

Principles

- ☐ **Stage the pipeline, keep raw immutable.** Model data in stages: **raw** → **normalized** → **enriched** → **filtered/validated** → **deliverable**. Never mutate raw scraped records — they are the re-runnable source of truth. Each stage reads the previous and writes a new layer.
- ☐ **Separate by tags, not by copies.** Rather than duplicating data into per-client or per-state tables, tag every record with the dimensions you'll slice on, and derive views from those tags:
 - ☐ `client_id` / ICP id
 - ☐ `record_type` (`contractor` | `vendor`)
 - ☐ `state`, `city`, `tier`, `zip`, `county`
 - ☐ `source` (`google_business` | `bbb` | `tn_license` | `nashville_open_data` | ...)
 - ☐ `scrape_run_id` + timestamp
 - ☐ `canonical_entity_id` (the dedup key — see below)
 - ☐ status flags: `out_of_territory`, `excluded_reason` (e.g. `lumber`), `enrichment_status`
- ☐ **Deduplicate via entity resolution.** Normalize name + address + phone to a `canonical_entity_id`, and link each source record to its canonical entity. This is also how GMS subsidiaries and L&W/ABC branches get merged. Keep the source rows; don't overwrite them.
- ☐ **Idempotent, re-runnable loads.** Key writes on (`canonical_entity_id`, `source`) with upserts so re-running a territory updates rather than duplicates. `scrape_run_id` makes every record traceable to a run.
- ☐ **Provenance / auditability.** For each record, store which run and source produced it, and (for enriched fields) where each value came from. This is what makes the lumber filter and the "wide net" relevance calls debuggable later.
- ☐ **Config and reference data are not scraped data.** Territory lists, city tiers, radii, the vendor alias map, and the negative-keyword list live in **separate, editable reference tables** — never hardcoded, never mixed into the results tables.

Access & export

- ☐ **Read through views/an API, not raw tables.** Downstream consumers (exports, CRM sync, dashboards) should read from **filtered views** parameterized by `client_id`, `record_type`, `territory`, and `tier` — never query raw directly.
- ☐ **Deliverables are derived, not stored filtered.** Generate the **two output lists** (contractors, vendors) as **derived exports/views** at request time, applying territory + lumber + tier filters there. Keep the underlying data wide (the "capture everything" requirement) and let the export do the narrowing.

- ☐ **Scope access per client.** Ensure one client's data cannot leak into another's export. If on Postgres/Supabase, row-level security or per-client schemas are reasonable approaches; the key requirement is enforced isolation at the access layer.
- ☐ **Handle contact data responsibly.** Scraped records contain personal contact info — apply access controls and a retention policy; don't expose raw contact data more broadly than needed.

Done when: the same underlying dataset can produce a clean, client-scoped, territory-scoped contractor list and vendor list on demand, with raw data preserved, duplicates merged, and every exclusion and source traceable.

3. Decisions needed before/while building

1. **Big-box anchors** — should Home Depot / Lowe's store locations be used as vendor anchors the same way specialty distributor branches are? (Relevant because the radius logic names Home Depot, and these chains now own GMS/FBM — risk of double-counting.)
 2. **Vendor roll-up** — confirm all GMS subsidiary brands roll up to a single "GMS" entity in output, and confirm the specific independent dealers to prioritize.
 3. **License-data depth** — start with Google Business + BBB + Nashville open data now and add the TDCI open-records roster later, or pursue the roster up front?
 4. **Tenancy model for data separation** — confirm whether this platform is single-client or expected to serve multiple clients, which determines how strict the per-client isolation in Workstream E needs to be.
-

Glossary

- **Apify** — the platform the scrapers run on.
- **ICP** — Ideal Customer Profile; the criteria defining a target lead.
- **DBPR** — Florida's contractor-license authority (Florida-only; stays for Florida).
- **TDCI** — Tennessee Department of Commerce & Insurance; its Board for Licensing Contractors handles TN licenses.
- **BBB** — Better Business Bureau.
- **Vendor / Dealer / Distributor** — used interchangeably; a business that sells drywall materials (e.g. ABC/L&W, FBM, GMS).
- **GC** — General Contractor. **Sub** — Subcontractor (the crew doing the actual drywall/taping).
- **Enrichment** — filling missing contact data after scraping.

- **Entity resolution / dedup** — merging multiple source records that refer to the same real business into one canonical entity.
- **Canonical entity** — the single merged record representing one real business across all sources.
- **Provenance** — the record of which run/source produced a given piece of data.